

操作系统

第二章 进程管理

0、程序的执行(最后一部分的讲稿)

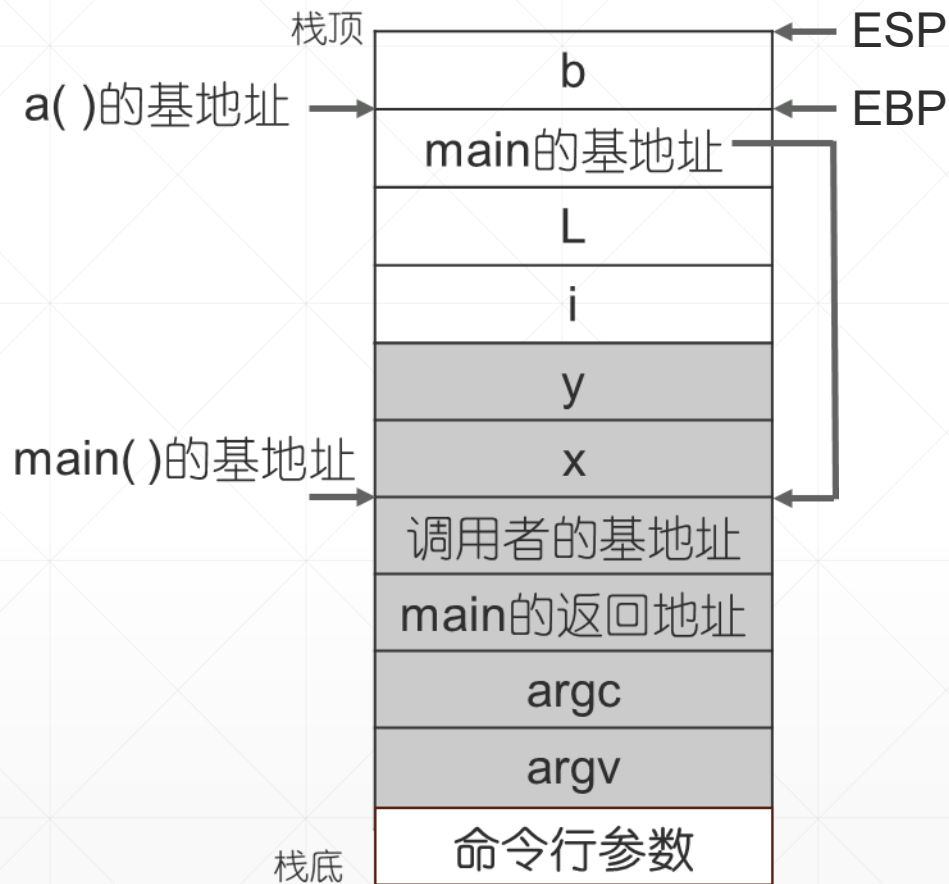
C程序的栈帧

```

int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    L: .....
}

int a( int i)
{
    int b = ++i;
    return b;
}
    
```

这是我们的应用程序，`main()`函数调用`a()`函数。`L`是`a()`的返回地址。

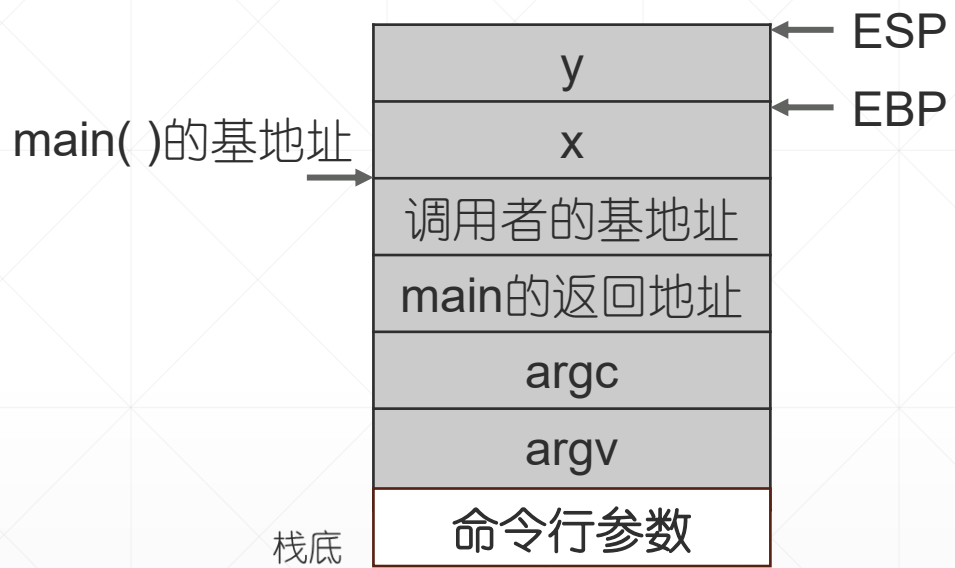


执行`a()`函数时，栈长这样。栈底放程序的命令行参数，灰色是`main()`栈帧，白色是`a()`栈帧。

寄存器 `ESP` 和 `EBP`，管理正在执行的函数栈帧，`a()` 栈帧。`ESP` 定位栈顶，`EBP` 用来访问这个子程序的入口参数和局部变量。具体而言：

局部变量：
[`EBP-4`], `b`

实参：
[`EBP+8`], `i`



执行main()函数时，栈长这样。栈底放程序的命令行参数，灰色是main()栈帧，白色是a()栈帧。

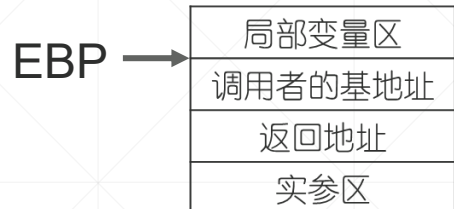
局部变量：
[EBP-4], x
[EBP-8], y

实参：
[EBP+8], argc
[EBP+12], argv



函数调用的汇编结果

C函数栈帧



局部变量的表示

[EBP-4], 第1个局部变量
[EBP-8], 第2个局部变量
.....

形参的表示

[EBP+8], 第1个形参
[EBP+12], 第2个形参
.....

main函数执行时

[EBP-4], x
[EBP-8], y
[EBP+8], argc
.....

```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
```

.....
L1: push -0x4(%ebp)
L2: **call a**
L12: add 0x4, %esp
L13: mov %eax, -0x8(%ebp)
.....

函数调用语句 **y = a(x)** 汇编结果

```
int a( int i)
{
    int b = ++i;
    return b;
}
```

L3: push %ebp
L4: mov %esp,%ebp
L5: sub \$0x4,%esp
L6: incl 0x8(%ebp)
L7: mov 0x8(%ebp),%eax
L8: mov %eax,-0x4(%ebp)
L9: mov -0x4(%ebp),%eax
L10: leave
L11: ret

子程序**a()**的汇编结果



附录：AT&T 汇编指令简介

`%ebp`, 百分号表示寄存器

`-0x4(%ebp)`, 这是用间接寻址方式表示的内存操作数。

`ebp`的值减4是一个内存单元的地址, 取这个内存单元的值。

`mov 0x8(%ebp),%eax` 源操作数在左, 目的操作数在右

我们知道`ebp+8`是`a()`的 `i` 变量。这条指令, `i`变量的值赋给`eax`寄存器

下面介绍子程序调用返回时, 堆栈构造和销毁过程。

汇编指令 (L1~L5) 调用a()子程序, 传参与 栈帧构造

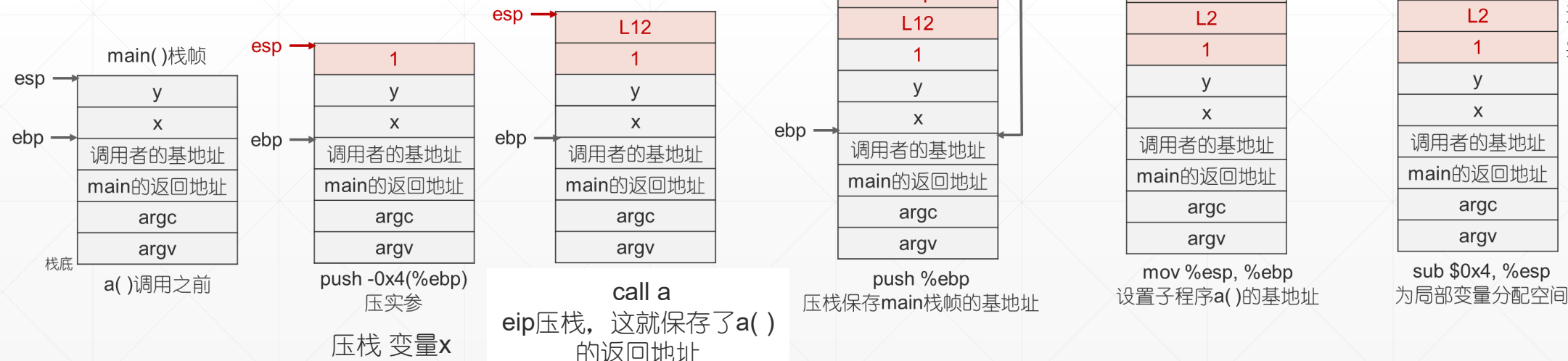
```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
```

.....

L1: push -0x4(%ebp)
 L2: call a
 L12: add 0x4, %esp
 L13: mov %eax, -0x8(%ebp)


```
int a( int i)
{
    int b = ++i;
    return b;
}
```

L3: push %ebp
 L4: mov %esp,%ebp
 L5: sub \$0x4,%esp
 L6: incl 0x8(%ebp)
 L7: mov 0x8(%ebp),%eax
 L8: mov %eax,-0x4(%ebp)
 L9: mov -0x4(%ebp),%eax
 L10: leave
 L11: ret



至此, `a()` 栈帧构造完成, CPU 可以基于 `ebp` 用间接寻址方式访问 `a()` 的实参和局部变量。

a()的运算, b = ++i

```
L6:  incl    0x8(%ebp)           // ++i
L7:  mov     0x8(%ebp),%eax      // i 的值赋给 eax寄存器
L8:  mov     %eax,-0x4(%ebp)     // eax寄存器的值赋给局部变量 b
```

return b 子程序返回的第一步, **eax**寄存器准备返回值

```
L9:  mov     -0x4(%ebp),%eax     // 局部变量 b的值赋给寄存器eax
```


子程序返回的第二步，销毁栈帧 (L10~L12)

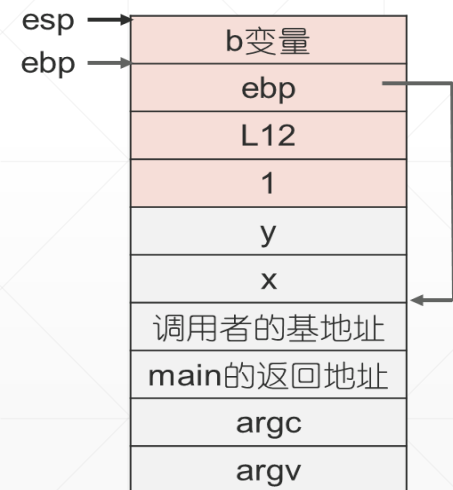
```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
```

```
.....
L1:  push -0x4(%ebp)
L2:  call a
L12: add 0x4, %esp
L13: mov %eax, -0x8(%ebp)
.....
```

```
int a( int i)
{
    int b = ++i;
    return b;
}
```

```
L3:  push %ebp
L4:  mov  %esp,%ebp
L5:  sub  $0x4,%esp
L6:  incl 0x8(%ebp)
L7:  mov  0x8(%ebp),%eax
L8:  mov  %eax,-0x4(%ebp)
L9:  mov  -0x4(%ebp),%eax
L10: leave
L11: ret
```

★ leave等价于
mov %ebp,%esp
pop %ebp



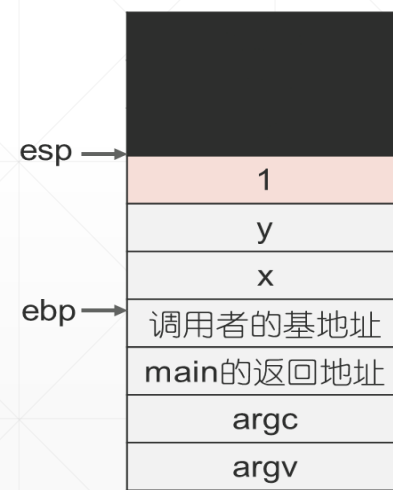
子程序返回前



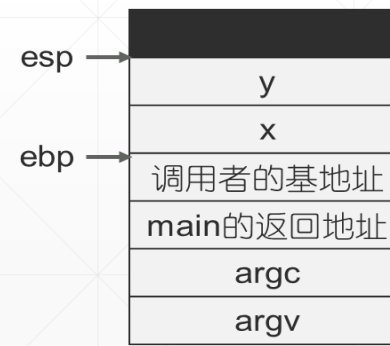
mov %ebp,%esp
清局部变量区



恢复main()的栈基



ret (eip = L12)
弹出栈顶保存的返回地址，赋值eip，返回main函数中的调用点



add 0x4, %esp
返回main()之后，清理a()栈帧的实参区

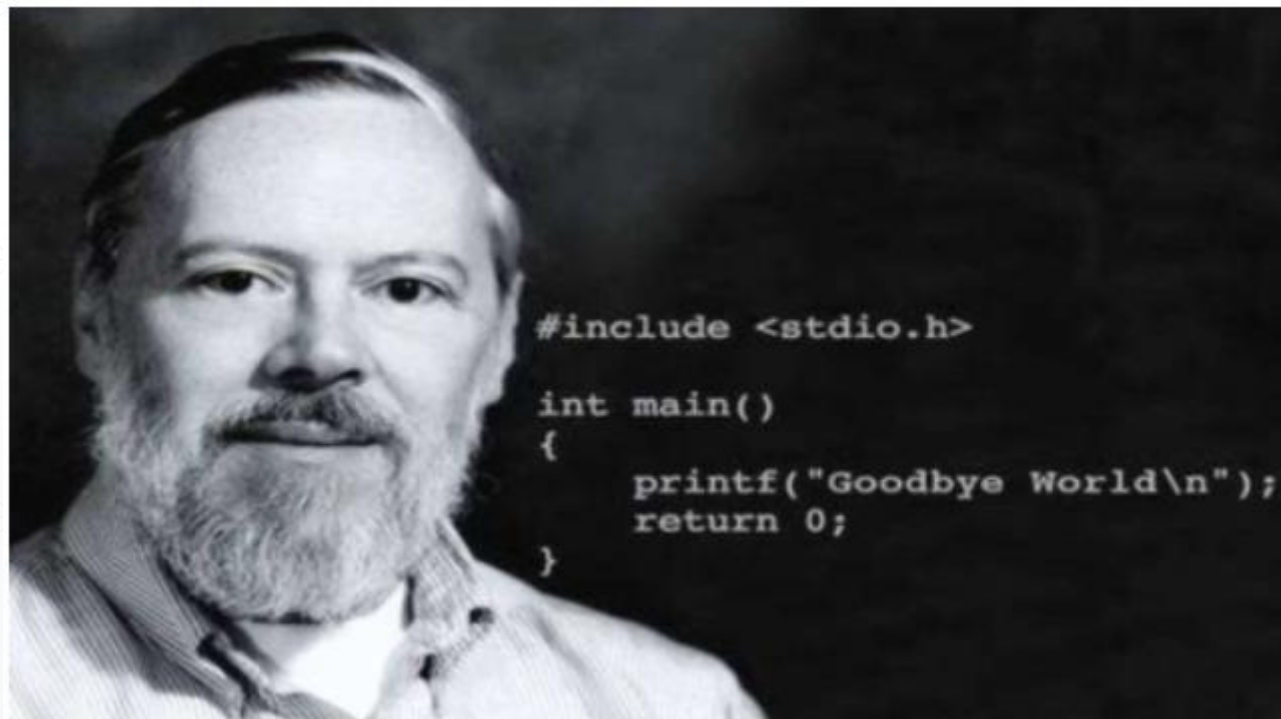
至此，a()栈帧清理完毕，恢复调用前的状态



无论局部变量有多少，一步到位，清理干净。好酷的设计啊

leave

C编译器在子程序的调用点，入口和返回处加上用来维护栈帧的代码。。。计算机成为了一台可以自动执行C子程序的机器。又一个伟大的设计！



可执行文件



4、可执行文件

- 每个可执行文件是一个程序。
- 格式:

程序头, 代码段, 数据段, 只读数据段, 符号表

- 程序头是一个数据结构, 描述程序地址空间中的逻辑段。

```
D:\UNIX V6++V1\oos\tools>objdump -h showStack.exe

showStack.exe:      file format pei-i386

Sections:
Idx Name          Size      VMA           LMA           File off  Algn
 0 .text          000020e8  00401000  00401000  00000400  2**2
   CONTENTS, ALLOC, LOAD, READONLY, CODE
 1 .data          0000000c  00404000  00404000  00002600  2**2
   CONTENTS, ALLOC, LOAD, DATA
 2 .rdata         00000204  00405000  00405000  00002800  2**3
   CONTENTS, ALLOC, LOAD, READONLY, DATA
 3 .bss           00000020  00406000  00406000  00000000  2**2
   ALLOC
```

展示子程序调用返回细节的ShowStack程序

```
#include <stdio.h>
int version=1;

main1()
{
    int a,b,result;
    a = 1;
    b = 2;
    result = sum(a,b);
    printf("result=%d\n",result);
}

int sum(var1, var2)
{
    int count;
    version = 2;
    count = var1 + var2;
    return(count);
}
```



这是在
初始化
程序的
所有逻
辑段

- 需要执行应用程序时，读可执行文件的程序头，得到程序的内存布局。
- 为代码段分配足够内存，磁盘读入可执行文件中的代码段。
- ~数据段~~~~~，磁盘读入可执行文件中的数据段（这是在为全局变量赋初值）。
- ~只读数据段~~~~~，~~~~~为常量赋值。
- 为bss分配内存，清0。 这是在为未赋值全局变量赋初值，0。
- ESP = 0x800000。
- push 压入命令行参数
- call main, 程序开始跑。

作业

- 实验二 Unix V6++ 程序栈帧和内存布局，观察可执行程序。